رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

Getting Started with Git and GitHub

Branching and Merging Strategies

- **Branching** and **merging** are fundamental concepts in version control systems like Git, allowing for parallel development and facilitating collaboration among team members.
- These strategies are essential for managing changes in a codebase effectively.

- Before diving into specific strategies, let's clarify the fundamental concepts:

Branch: A separate line of development from the main project.

Merge: Combining changes from one branch into another.

Fork: A copy of a repository that allows independent development.

Branching

- Branching in Git allows developers to create separate lines of development for new features, bug fixes, or experiments.
- Each branch is an independent line of development, allowing for changes to be made **without** affecting the main codebase until they are ready to be integrated.

Creating a Branch:

To create a new branch in Git, you use the **git branch** command followed by the name of the branch:
git branch <branch-name>.
This command creates a new branch from your current branch.

To start working on this new branch, you need to switch to it using the **git checkout** command:
git checkout <branch-name>

Git also provides a shorthand command to **create** and **checkout** a new branch in one step:
git checkout -b <branch-name>

Naming Conventions:
It's a good practice to use descriptive names for branches, indicating the purpose or feature they are associated with (e.g., feature/user-authentication, bugfix/login-issue).

Merging

- **Merging** is the process of integrating the changes from one branch back into another, typically, the **main branch** (often called **master** or **main**).
- This is how completed features or bug fixes make their way into the main codebase.

Merging a Branch:

- To merge a branch into your current branch, you first need to checkout the branch you want to merge into (e.g., master or main), and then use the **git merge** command:
- **git checkout main**
- **git merge <branch-name>**
- This command will integrate the changes from **<branch-name>** into the **main** branch. If there are no conflicts, Git will perform a fast-forward merge or create a merge commit if necessary.

Merge Conflicts:

- Sometimes, changes in the two branches may conflict with each other.
  In such cases, Git will not be able to automatically merge the branches and will require manual intervention to resolve the conflicts.
  After resolving the conflicts, you need to mark the files as resolved and complete the merge.

Strategies

There are several branching and merging strategies that teams can adopt, depending on the size of the team, the complexity of the project, and other factors. Some popular strategies include:

Feature Branch Workflow: Develop new features in separate branches and merge them back into the main branch once they are complete and reviewed.

Git Flow: A more complex workflow that defines a strict branching model designed around the project release.
It involves multiple types of branches for features, releases, hotfixes, and maintenance.

Forking Workflow: Contributors fork the main repository, develop and merge changes in their forks, and then submit pull requests to the original repository.

Choosing the right strategy depends on the specific needs and workflows of your team. Regardless of the strategy, branching and merging provide a powerful way to manage changes in a codebase, ensuring that development can proceed smoothly and collaboratively.

What are Merge Conflicts?

- Merge conflicts occur in version control systems like Git when two branches have made changes to the same lines of the same file, and the system is unable to automatically reconcile these changes.
- This typically happens when multiple people are working on the same project or when an individual is working on multiple branches simultaneously.

- Merge conflicts arise because Git is designed to track changes to the codebase over time.
  When two sets of changes cannot be automatically merged, Git requires human intervention to decide which changes should be kept, modified, or discarded.
- This ensures that no work is lost and that the integrity of the codebase is maintained.

Resolving Conflicts

- Resolving merge conflicts is a critical skill for anyone working with Git. Here are some strategies for resolving conflicts effectively:
- **Understand the Conflict**: Before resolving a conflict, it's important to understand what changes are in conflict. Git provides conflict markers in the affected files that show the changes from both branches. Review these changes to understand the context and what each change is intended to achieve.
- **Manual Resolution**: The most straightforward way to resolve a conflict is to manually edit the files. Remove the conflict markers and decide which changes should be kept. This might involve choosing one version over the other, combining changes, or rewriting the code to incorporate both sets of changes in a new way.
- **Use Merge Tools:** Many integrated development environments (IDEs) and text editors come with built-in merge tools or have extensions that can help visualize and resolve conflicts more easily.
  These tools can provide a side-by-side comparison of the conflicting changes, making it easier to decide how to resolve the conflict.
- **Communicate with Your Team:** If the conflict involves changes made by different team members, it's often helpful to discuss the changes directly with those involved.
  This can provide additional context and help ensure that the resolution aligns with the project's goals and that everyone's work is respected.
- **Test Your Resolution:** After resolving the conflicts, it's important to test the affected parts of the project to ensure that the resolution works as expected and hasn't introduced any new bugs.
- **Commit the Resolution:** Once you've resolved the conflicts and confirmed that everything works correctly, you can commit the changes. Include a clear commit message that mentions the conflict resolution, so it's easy for others to understand what happened.
- **Avoid Conflicts:** While conflicts are a natural part of collaborative development, there are strategies to minimize their occurrence.
  This includes frequent integration of changes, working on separate sections of the codebase, and using feature toggles to hide in-progress features.
- Resolving merge conflicts is an inevitable part of working with Git, especially in collaborative environments.
  By understanding why conflicts occur and employing effective strategies to resolve them, you can maintain the integrity of your codebase and ensure smooth collaboration with your team.

GitHub Features: Issues

- **Issues** on GitHub are used to track bugs, propose enhancements, and discuss tasks relating to a project's repository.
- They provide a simple but powerful system for organizing and prioritizing the work that needs to be done.
- **Creating an Issue:** To create an issue, navigate to the repository's Issues tab and click the "New issue" button. You can provide a title and description for the issue, and you can also assign labels, milestones, and assignees to help organize and track the issue.
- **Labels:** Labels are colored tags that can be applied to issues and pull requests to categorize them.
  Common labels include "**bug**," "**enhancement**," "**question**," and "**help wanted**."
- **Milestones:** Milestones track issues and pull requests that are part of a larger project goal. They help you manage progress towards that goal by giving you a way to track issues and pull requests that are associated with it.
- **Assignees:** Assignees are the people responsible for addressing the issue.
  You can assign issues to yourself or to other collaborators on the project.

GitHub Features: Pull Requests

- A **Pull Request** (**PR**) is a method of submitting contributions to a project.
  It shows diffs (differences) of the content from both branches.
  The changes, additions, and subtractions are shown in green and red.
  It allows you to tell others about changes you've pushed to a branch in a repository on GitHub.
- **Creating a Pull Request:** To create a pull request, you need to have made some changes in a branch.
  Navigate to your fork or branch, and click on the "New pull request" button.
  GitHub will show you a comparison between your branch and the original repository's branch. If the diffs look good, you can submit a pull request.
- **Review and Discussion:** Once a pull request is opened, team members can review the changes, discuss potential improvements, and even push follow-up commits if they have the necessary permissions.
- **Merge:** After the review process, if the changes are approved, the pull request can be merged into the main branch.

Example

There are 2 main workflows when dealing with pull requests:

Pull Request from a forked repository

Pull Request from a branch within a repository

Here we are going to focus on 2.

Creating a Topical Branch

First, we will need to create a branch from the latest commit on master.
Make sure your repository is up to date first, using

```
git pull origin master
```

Note: **git pull** does a **git fetch** followed by a **git merge** to update the local repo with the remote repo.

To create a branch, use **git checkout -b <new-branch-name> [<base-branch-name>]**,
where **base-branch-name** is optional and defaults to master.
I'm going to create a new branch called **pull-request-demo** from the **master** branch and push it to GitHub.

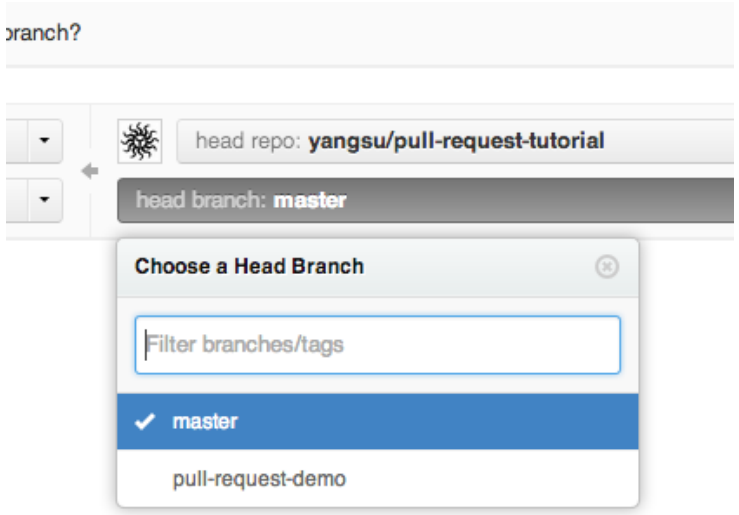
```
git checkout -b pull-request-demo
git push origin pull-request-demo
```

Creating a Pull Request

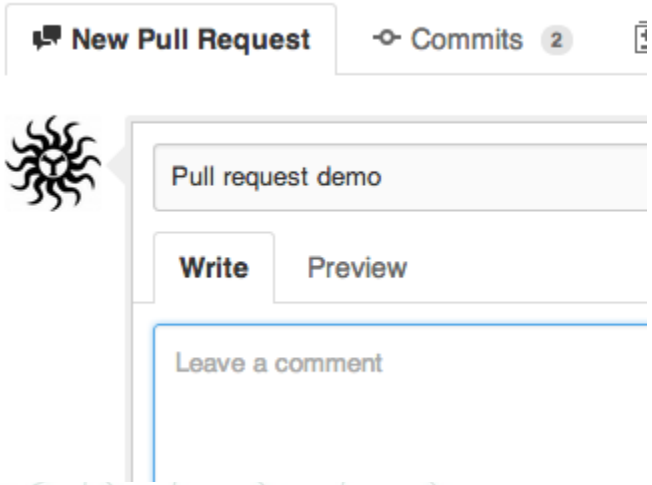
To create a pull request, you must have changes committed to your new branch.

Go to the repository page on GitHub. And click on the "Pull Request" button in the repo header.

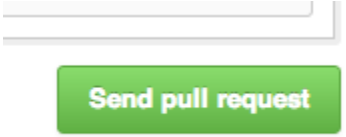
Pick the branch you wish to have merged using the "Head branch" dropdown.
You should leave the rest of the fields as is, unless you are working from a remote branch.
In that case, just make sure that the base repo and base branch are set correctly.

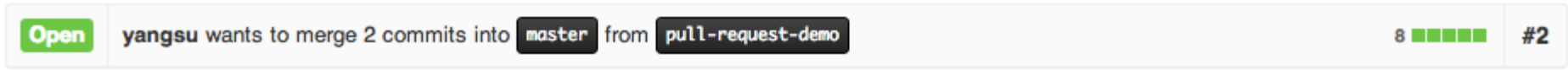


Enter a title and description for your pull request.
Remember, you can use GitHub Flavored Markdown in the description and comments



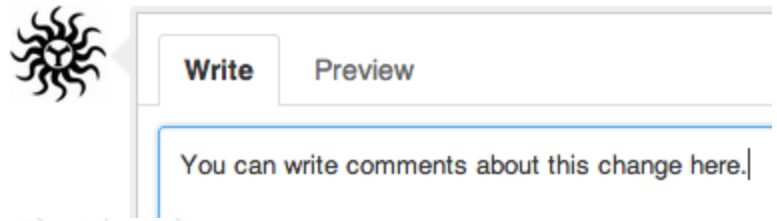
Finally, click on the green "Send pull request" button to finish creating the pull request.




Open
yangsu wants to merge 2 commits into master from pull-request-demo
8
#2
Discussion
Commits 2
Files Changed 1
yangsu opened this pull request 2 minutes ago
Pull request demo
Edit
No one is assigned
No milestone
This is a demo of how to do a pull request!
1 participant
Open
+ 8 additions
- 0 deletions
yangsu added some commits
20 minutes ago
yangsu started the creating a pull request section 4782b63
yangsu Added first step in creating a pull request f356b3c
You can add more commits to this pull request by pushing to the pull-request-demo branch on yangsu/pull-request-tutorial
This pull request can be automatically merged.
Merge pull request

Using a Pull Request

You can write comments related to a pull request,

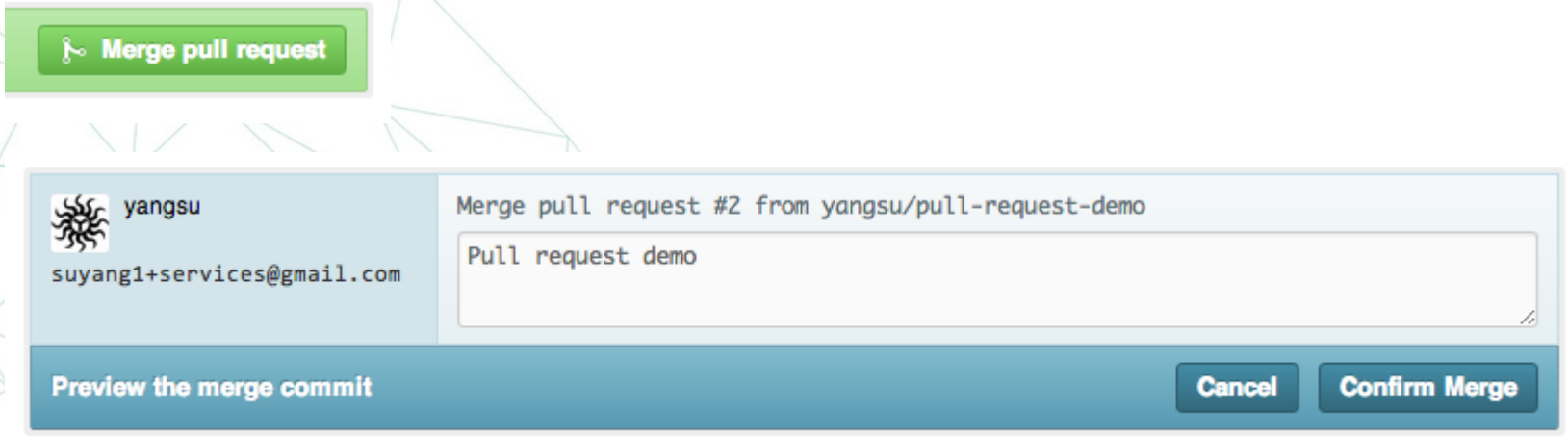


view all the commits by all contained by a pull request under the commits tab,

Merging a Pull Request

Once you and your collaborators are happy with the changes, you start to merge the changes back to master.
There are a few ways to do this.

First, you can use GitHub's "Merge pull request" button at the bottom of your pull request to merge your changes.
This is only available when GitHub can detect that there will be no merge conflicts with the base branch.
If all goes well, you just have to add a commit message and click on "Confirm Merge" to merge the changes.



GitHub Features: Forking

- **Forking** is a feature on GitHub that allows you to create a copy of a repository to your own account without affecting the original repository.
  This is useful for proposing changes to someone else's project or for trying out something in a safe space.

- Forking a Repository: To fork a repository, navigate to the repository you wish to fork and click the "Fork" button. GitHub will create a copy of the repository in your account.

- Making Changes: After forking, you can clone the forked repository to your local machine, make changes, and push them to your fork.

- Submitting Changes: If you want to contribute your changes back to the original repository, you can submit a pull request from your fork to the original repository.

Collaborating with Teams on GitHub

- Collaborating with teams on GitHub is a fundamental aspect of the platform's design, allowing multiple individuals to work together on a project efficiently.
- Here's how to add and manage collaborators on a repository and understand and configure team permissions.
- Collaborators are individuals who have been invited to contribute to a repository. They can push changes directly to the repository, create new branches, and more, depending on the permissions granted to them.
- In addition to individual collaborators, you can also manage access to repositories through teams, especially if your repository is part of an organization on GitHub.
- By effectively managing collaborators and configuring team permissions, you can ensure that your project on GitHub is accessible to the right people with the appropriate levels of access, fostering a collaborative and secure environment for software development.

Adding Collaborators

- To add a collaborator to a repository:
- **Navigate to the Repository:** Go to the main page of the repository where you want to add a collaborator.
- **Access Settings:** Click on "Settings" in the repository menu.
- **Manage Access:** In the sidebar, under "Code and automation," click on "Manage access."
- **Invite a Collaborator:** Click on "Add people" or "Invite a collaborator," depending on the repository's settings.
- **Search for Users:** Type the username or email address of the person you want to invite.
- **Select Permission Level:** Choose the appropriate permission level for the collaborator. The options typically include "Read," "Triage," "Write," "Maintain," and "Admin."
- **Send Invitation:** Click "Add NAME to repository" to send the invitation. The invited user will receive an email notification and can accept the invitation to become a collaborator.

Managing Collaborators

To manage existing collaborators:

- Access Manage Access: Follow the steps above to reach the "Manage access" section.
- Review Collaborators: You'll see a list of current collaborators and their permission levels.
- Modify Permissions: Click on the dropdown menu next to a collaborator's name to change their permission level or remove them from the repository.
- Remove Collaborators: To remove a collaborator, click "Remove" in the dropdown menu. Confirm the removal to update the collaborator's access.

Understanding Team Permissions

Team permissions determine what actions team members can take within the repository.

The permission levels for teams are similar to those for individual collaborators:

- **Read:** Can pull, clone, and view repository contents.
- **Triage:** Can do everything readers can, plus manage issues and pull requests without write access.
- **Write:** Can do everything triage can, plus push to the repository and manage branches and tags.
- **Maintain:** Can do everything writers can, plus manage collaborators, deploy keys, and repository settings.
- **Admin:** Full access to the repository, including the ability to delete it.

Configuring Team Permissions

- To configure team permissions:

1) Navigate to the Organization: Go to your organization's page.
2) Access Teams: Click on "Teams" in the organization's menu.
3) Create or Select a Team: Click "New team" to create a new team or select an existing team to modify.
4) Set Permissions: When creating a new team, you'll be prompted to set the team's permissions for repositories. Choose the appropriate permission level.
5) Add Repositories: Add the repositories that the team should have access to, along with the permission level for each repository.
6) Add Members: Invite members to join the team. They will inherit the team's permissions for the associated repositories.

Introduction to CI/CD

- Continuous Integration (CI) and Continuous Deployment (CD) are practices within the software development lifecycle that focus on automating the integration, testing, and deployment of code changes.
- These practices aim to improve software quality and reduce the time it takes to go from writing code to deploying it to production.

- Continuous Integration (CI): In CI, developers frequently integrate their code into a central repository. Each integration is verified by an automated build and test process to detect errors quickly and locate them more easily.

- Continuous Deployment (CD): CD automates the release of code to production. After the CI process successfully builds and tests the code, CD ensures that the deployment process is repeatable and reliable, reducing the risk of human error.

GitHub Actions

- GitHub Actions is a platform for automating workflows that can be triggered by various events on your GitHub repositories, such as pushes, pull requests, issues, and more.
- It allows you to write individual tasks (called "actions") that can be combined into custom workflows to support your CI/CD pipelines, among other things.
- To set up a CI/CD workflow using GitHub Actions, follow these steps:

1) **Create a Workflow File:** Workflows are defined in YAML files within the .github/workflows directory of your repository. For example, you might create a file named ci-cd.yml.
2) **Define the Workflow Trigger:** Specify the event that will trigger your workflow. Common triggers for CI/CD include push to a branch or the opening of a pull_request.
3) **Configure Jobs:** Define the jobs that make up your workflow. A **job** is a series of steps that execute on the same runner. For a CI/CD workflow, you might have jobs for building, testing, and deploying your application.
4) **Use Actions:** Utilize existing actions from the GitHub Marketplace or create your own to perform steps within your jobs. Actions can handle tasks like checking out code, installing dependencies, running tests, and deploying your application.
5) **Commit and Push:** Commit the workflow file to your repository and push it to your GitHub repository.
6) **Monitor Your Workflow:** Once your workflow file is in place, GitHub Actions will automatically run your workflow when the specified events occur. You can monitor the progress and results of your workflows in the "Actions" tab of your GitHub repository.

Exploring GitHub's Security Features

- GitHub offers a range of security features designed to help developers maintain the security of their codebases.
- Two important aspects of this are secure coding practices and the use of Dependabot for automated dependency updates and security alerts.
- **Dependabot** is a tool integrated into GitHub that helps keep your project's dependencies up to date and secure.
- It automatically creates pull requests to update your dependencies when a new version is released, including versions that fix security vulnerabilities.

Secure Coding Practices

- Secure coding practices are essential for developing software that is resilient against security vulnerabilities:
- **Input Validation:** Always validate user input to prevent injection attacks.
- **Authentication and Authorization:** Implement strong authentication and authorization mechanisms.
- **Error Handling:** Handle errors gracefully and **avoid exposing sensitive information**.
- **Session Management:** Manage sessions securely to prevent session hijacking.
- **Cryptographic Controls:** Use cryptographic mechanisms appropriately for data protection.
- **Secure Data Storage:** Store sensitive data securely, often using encryption.
- **Least Privilege:** Apply the principle of least privilege to user accounts and processes.
- **Regular Audits:** Conduct regular security audits and code reviews.
- **Stay Informed:** Keep up-to-date with the latest security trends and vulnerabilities.

Advanced Git Techniques: Rebasing

- Rebasing is a process that integrates changes from one branch into another by moving or combining a sequence of commits to a new base commit.
- This is often done to maintain a cleaner project history compared to merging, which can create a more complex history with merge commits.
- When to Rebase

Before Sharing Work: Rebase your feature branch onto the main branch before pushing your changes.
This ensures your branch is up to date with the latest changes in the main branch and avoids unnecessary merge commits.

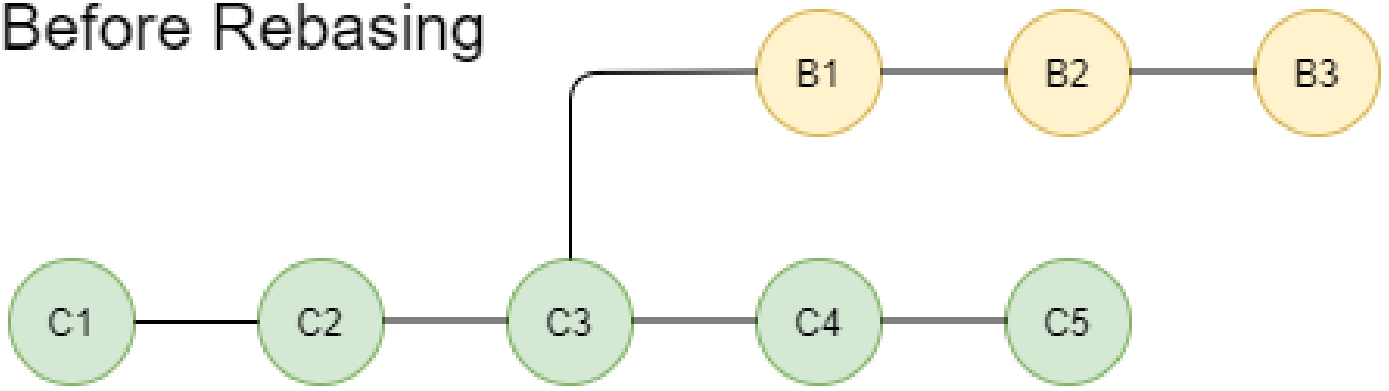
Cleaning Up History: Use rebasing to clean up your commit history before merging into the main branch.
This can involve squashing commits (combining smaller commits into larger ones) or rewording commit messages.

- How to Rebase

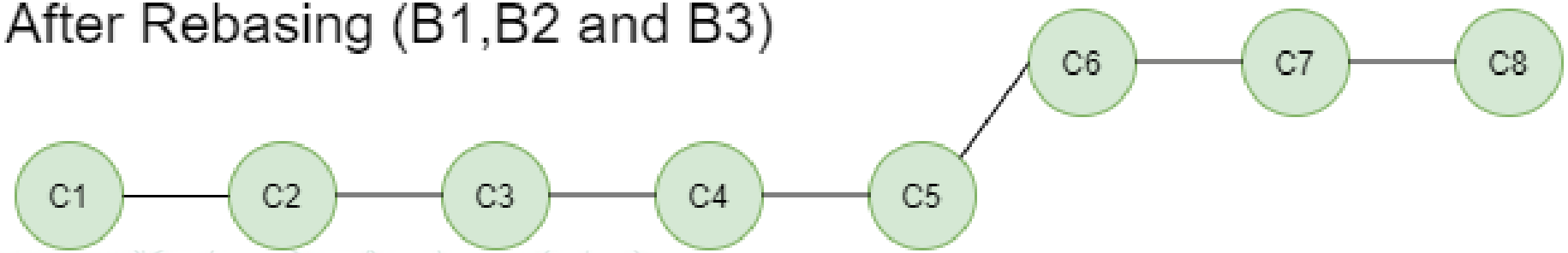
To rebase your current branch onto the main branch, use the following command:

- git checkout your-feature-branch
- git rebase main
- This will reapply your commits to the top of the main branch.
  If there are any conflicts, Git will pause and allow you to resolve them before continuing the rebase process.

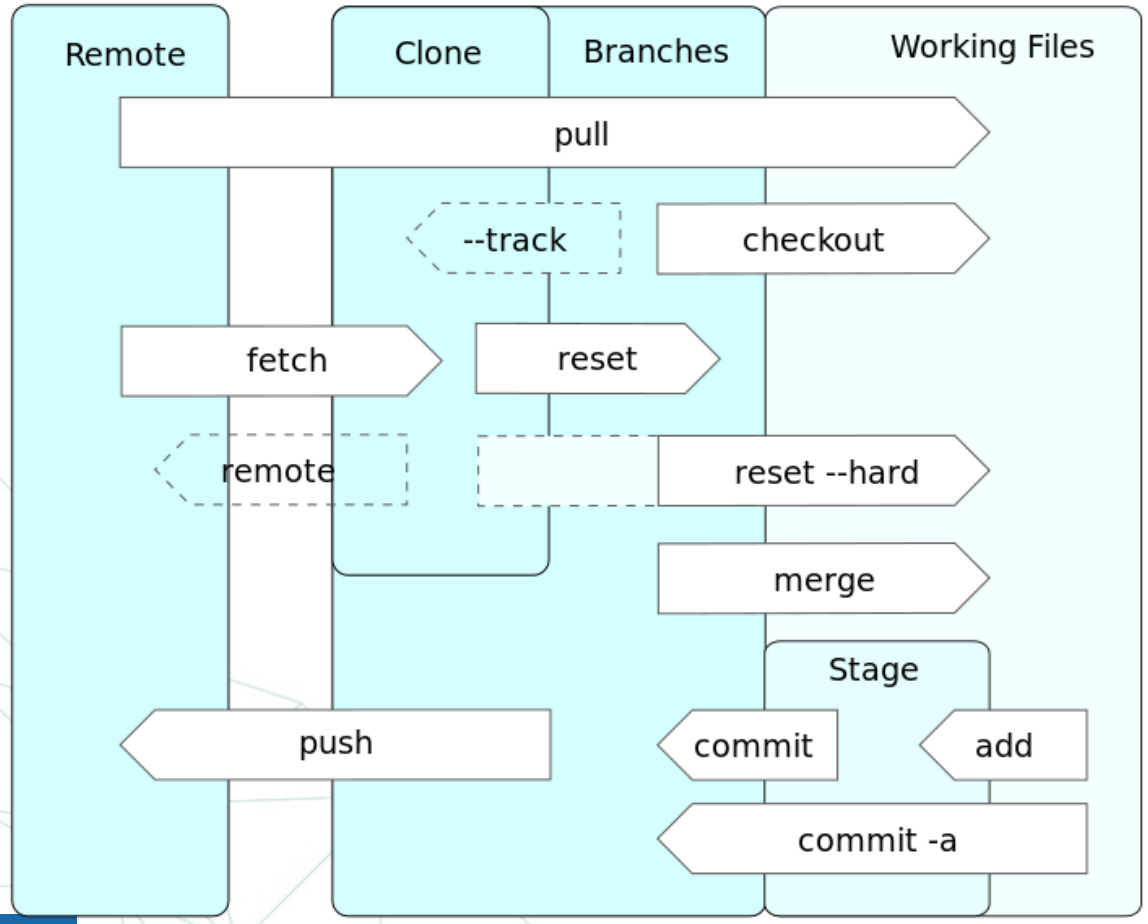
Before Rebasing



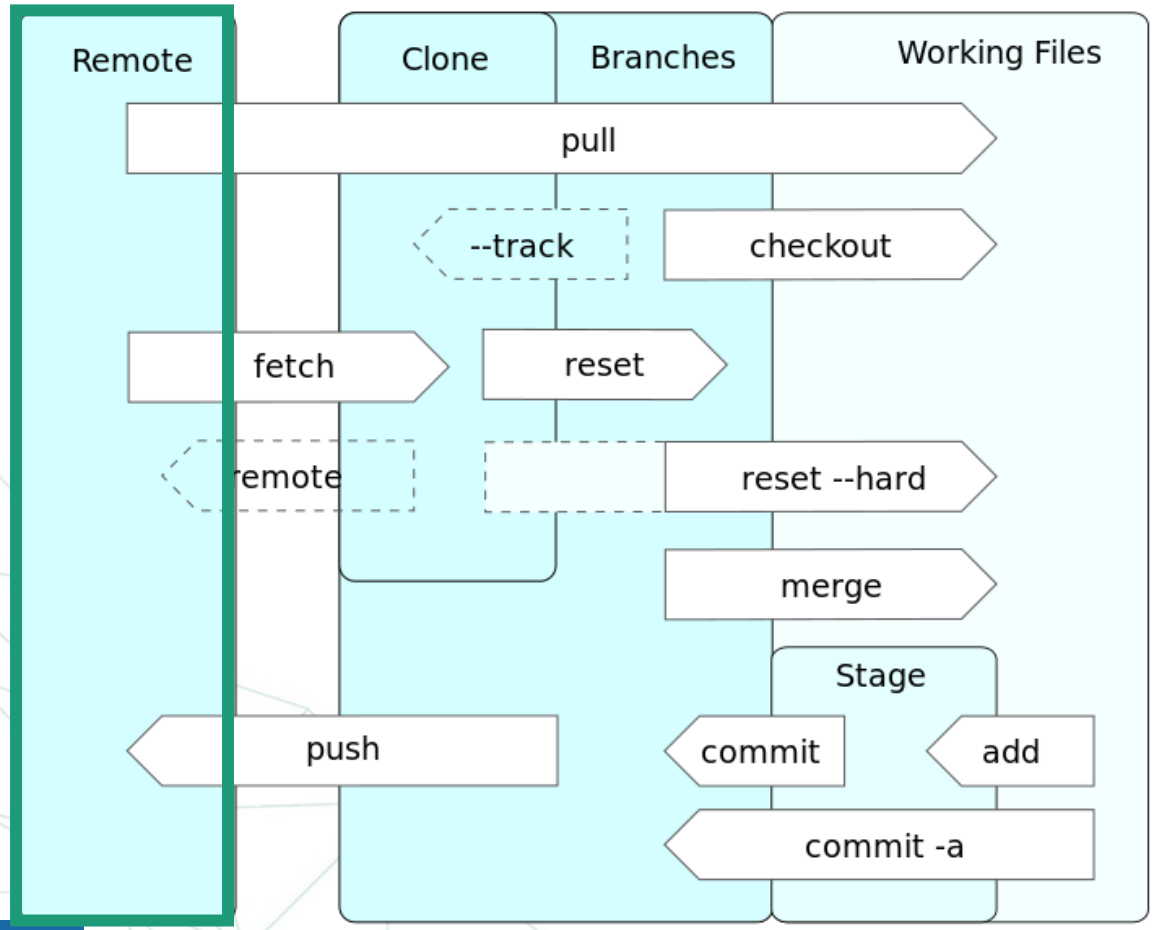
After Rebasing (B1,B2 and B3)



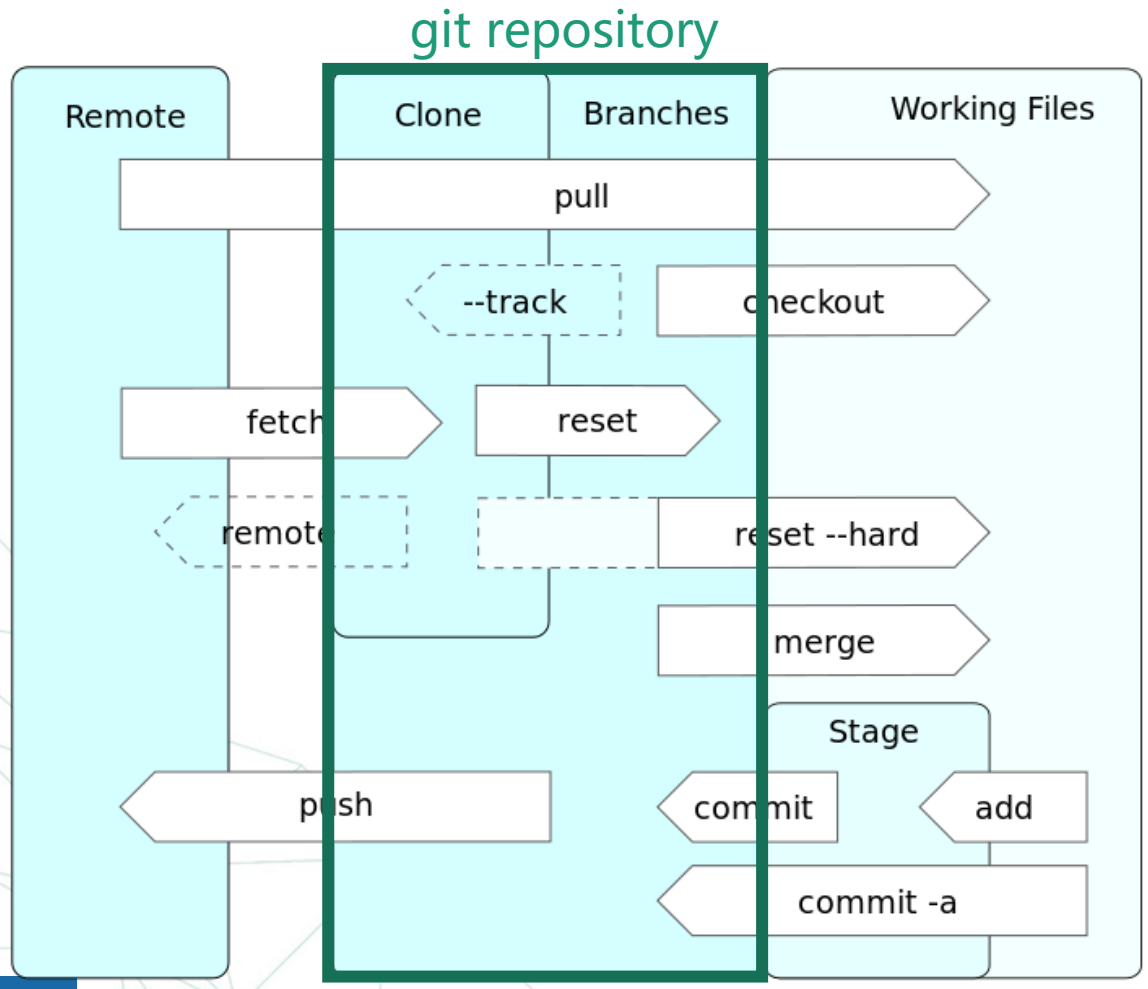
Operations


Remote
Clone
Branches
Working Files
pull
--track
checkout
fetch
reset
remote
reset --hard
merge
Stage
push
commit
add
commit -a

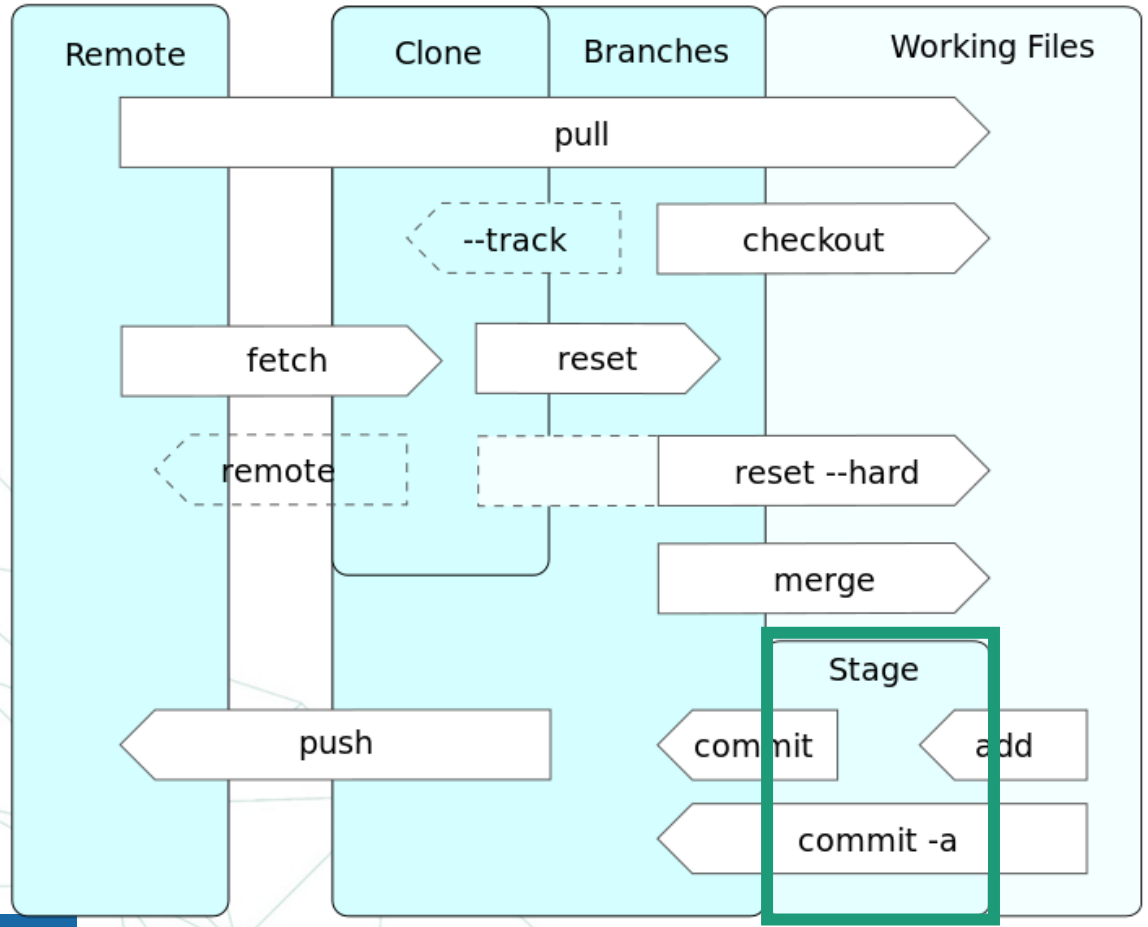
Operations


Remote
Clone
Branches
Working Files
pull
--track
checkout
fetch
reset
remote
reset --hard
merge
Stage
push
commit
add
commit -a

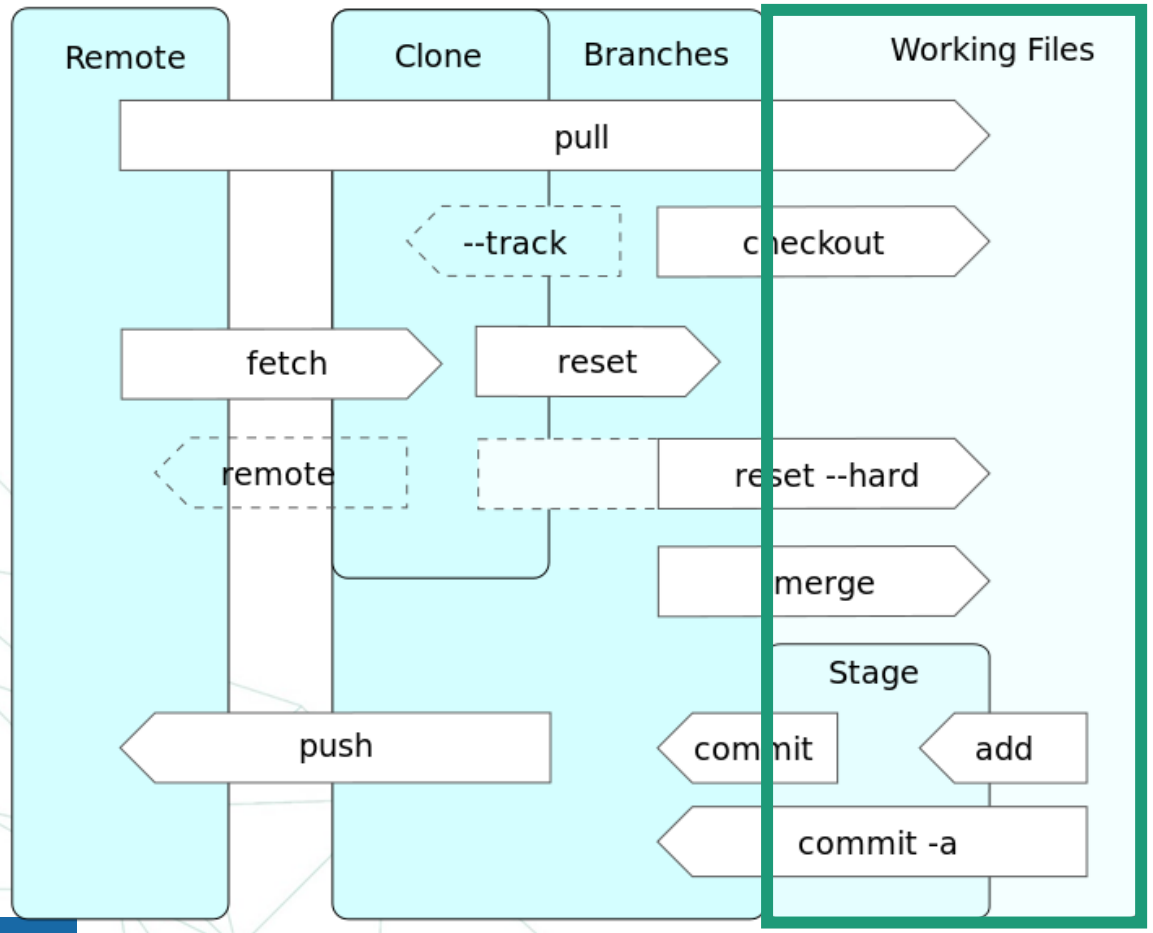
Operations


git repository
Remote
Clone
Branches
Working Files
pull
--track
checkout
fetch
reset
remote
reset --hard
merge
Stage
push
commit
add
commit -a

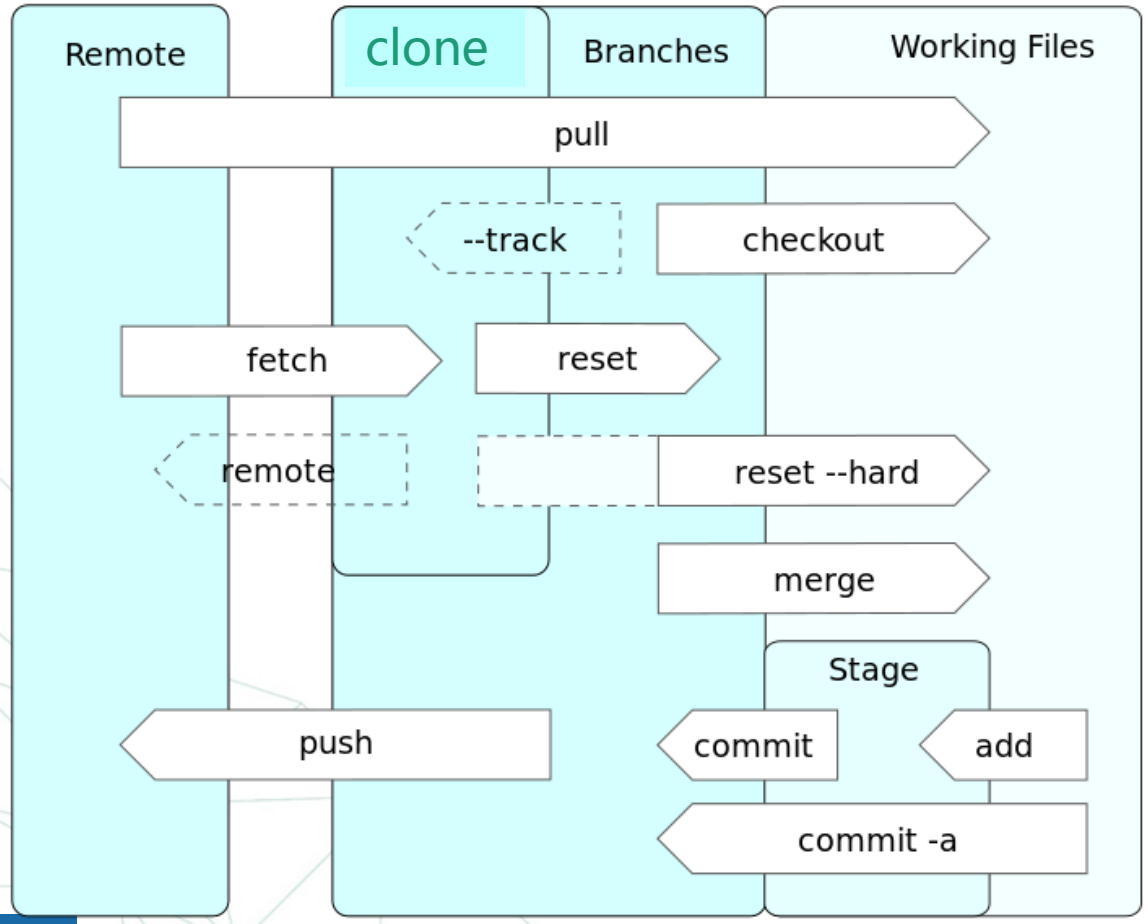
Operations


Remote
Clone
Branches
Working Files
pull
--track
checkout
fetch
reset
remote
reset --hard
merge
Stage
push
commit
add
commit -a

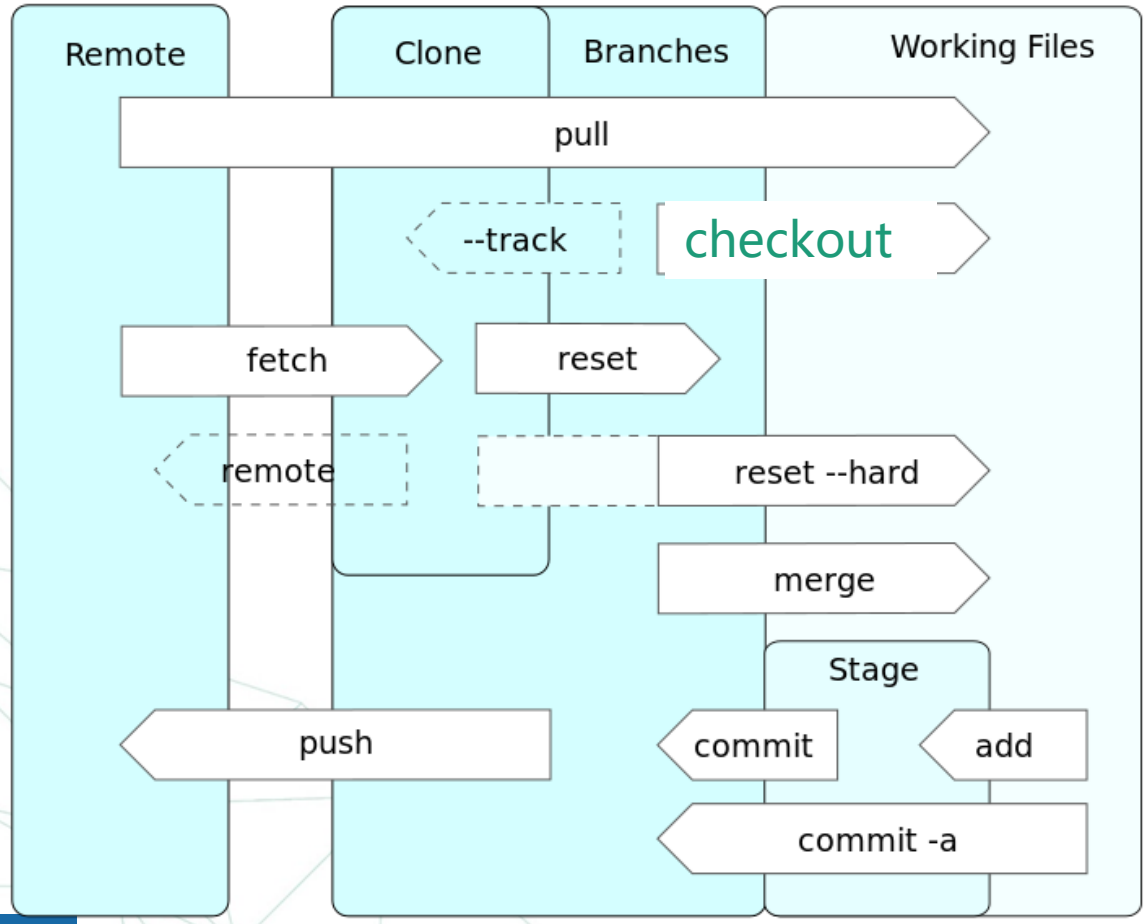
Operations


Remote
Clone
Branches
Working Files
pull
--track
checkout
fetch
reset
remote
reset --hard
merge
Stage
push
commit
add
commit -a

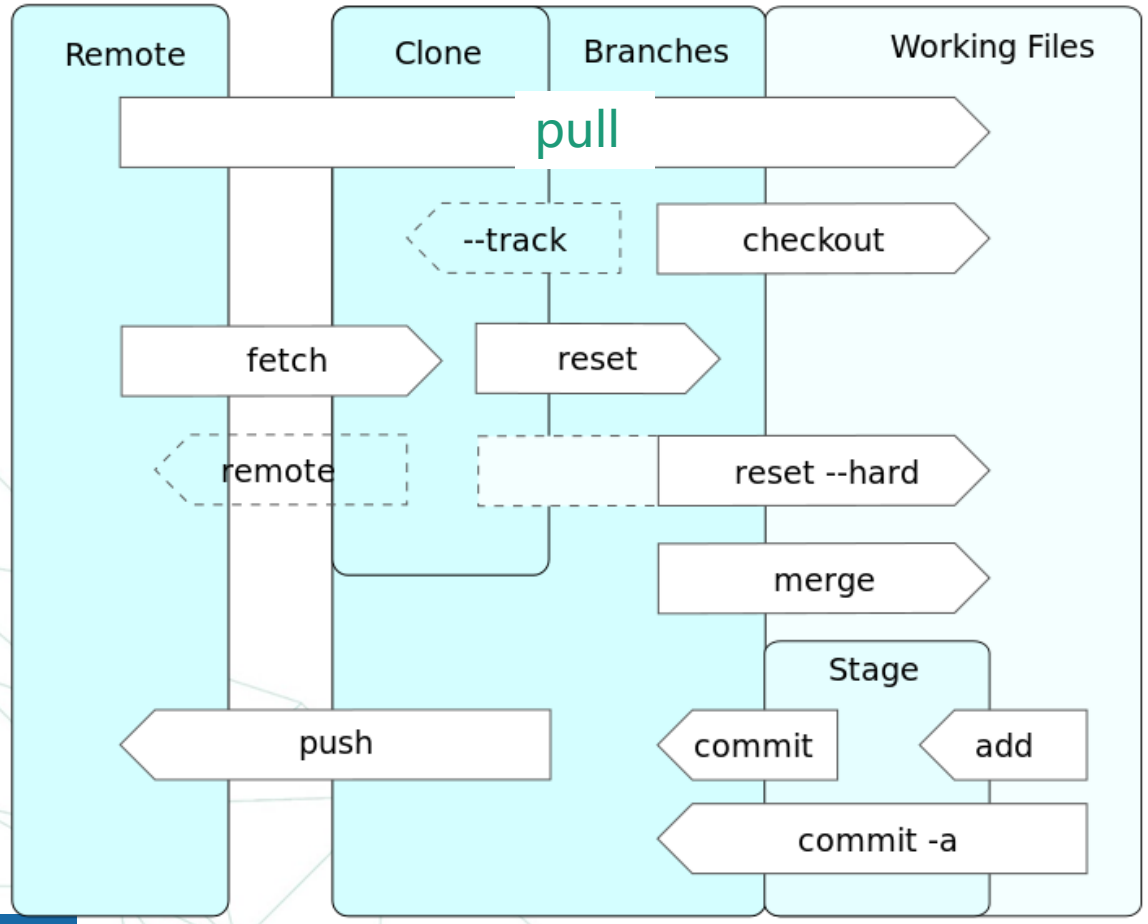
Operations


Remote
clone
Branches
Working Files
pull
--track
checkout
fetch
reset
remote
reset --hard
merge
Stage
push
commit
add
commit -a

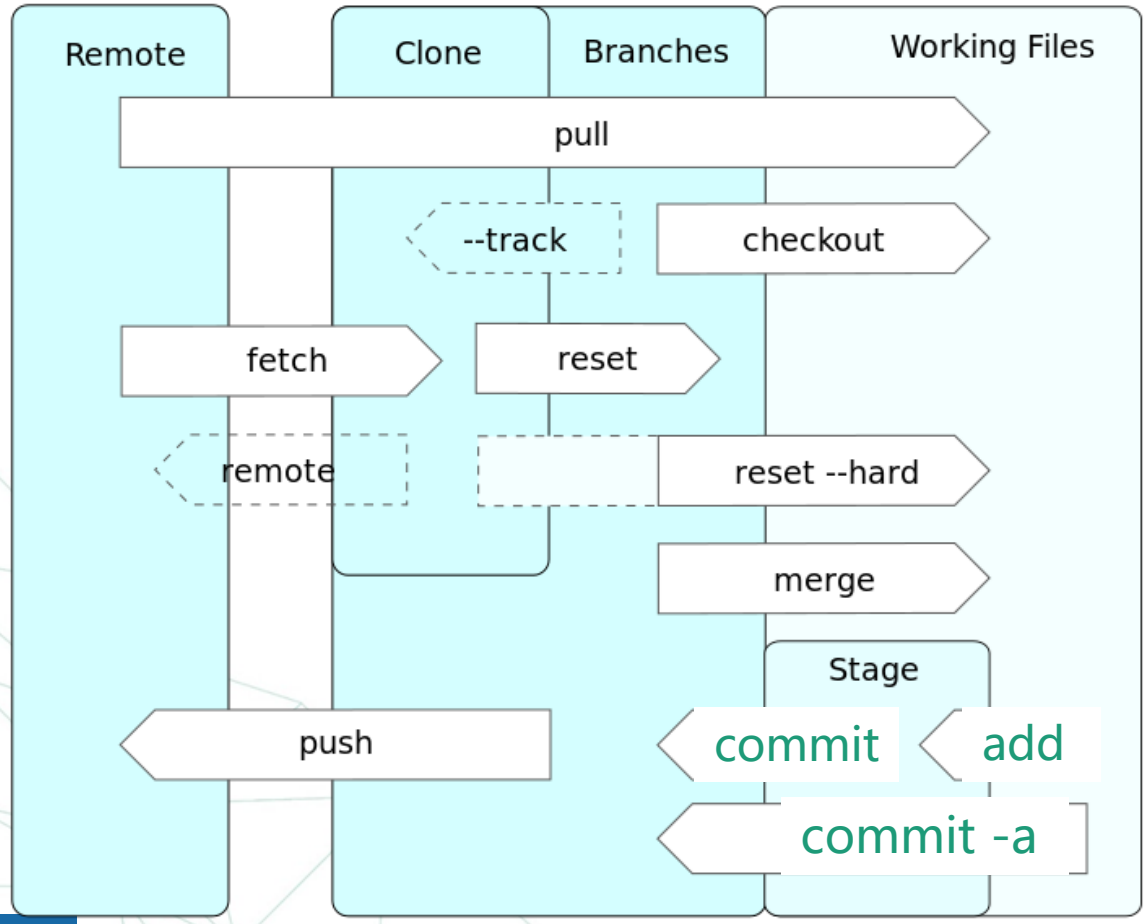
Operations


Remote
Clone
Branches
Working Files
pull
--track
checkout
fetch
reset
remote
reset --hard
merge
Stage
push
commit
add
commit -a

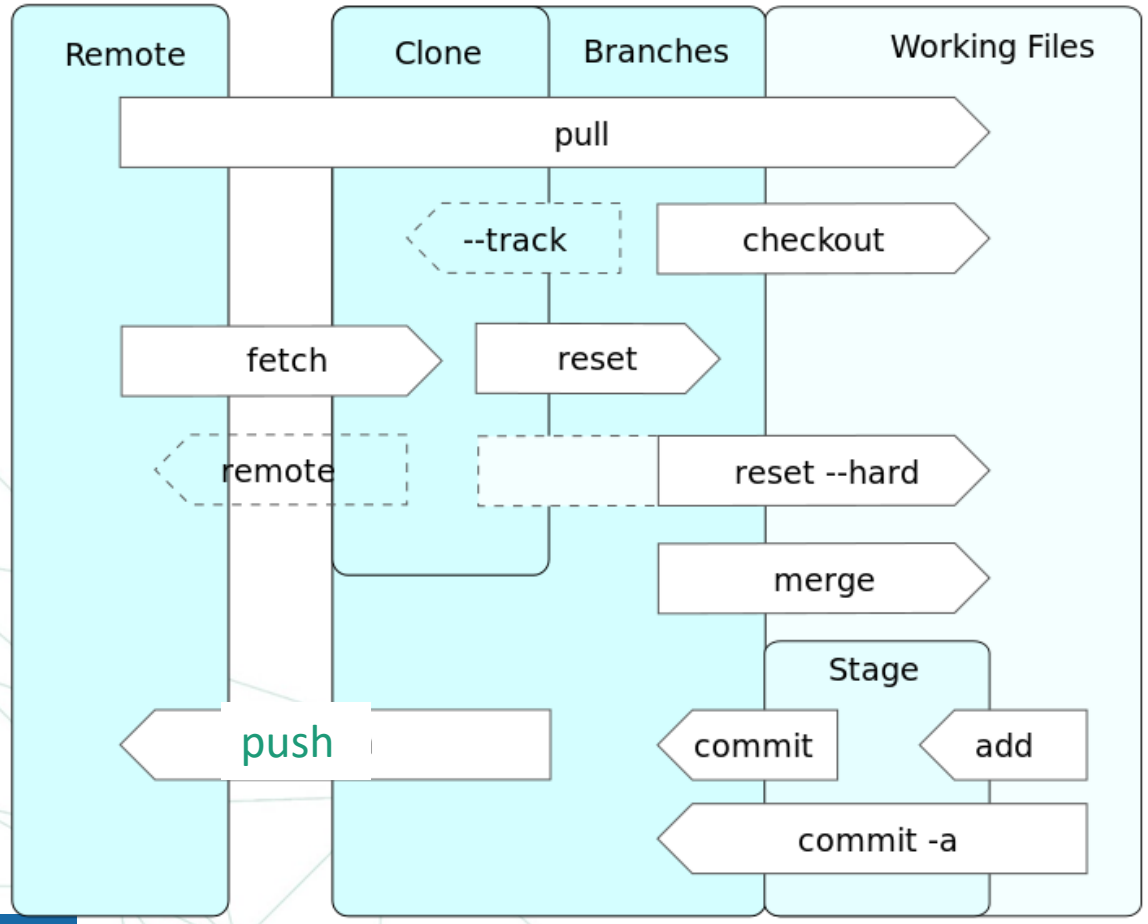
Operations


Remote
Clone
Branches
Working Files
pull
--track
checkout
fetch
reset
remote
reset --hard
merge
Stage
push
commit
add
commit -a

Operations


Remote
Clone
Branches
Working Files
pull
--track
checkout
fetch
reset
remote
reset --hard
merge
Stage
push
commit
add
commit -a

Operations


Remote
Clone
Branches
Working Files
pull
--track
checkout
fetch
reset
remote
reset --hard
merge
Stage
push
commit
add
commit -a

What GitHub Does

- Online project hosting site.
- "Remote" git repository with access control
- Issue Tracking
- Documentation and web pages (github.io)
- Integrates with other services
  - Continuous Integration, e.g. CircleCI

What to do

Create a GitHub Account

- Put your REAL NAME in profile
- Add a PHOTO that clearly shows your face
- Include a public Email. Prefer KU-Gmail
- Write a short profile about yourself

GitHub Account

Students in last year's OOP course.

1. Real name
2. Photo
3. Email
4. Description of you

Jirayu Laungwilawan
JirayuL

Faculty of Engineering , Major - Software and Knowledge Engineering.

Follow

Block or report user

Thailand
jirayu.l@ku.th
https://github.com/JirayuL

Kongpon Charanwattanakit
kykungz

Software Developer, Undergraduate Software and Knowledge Engineering Student

Follow

Block or report user

Kasetsart University
Bangkok, Thailand
jackykongpon@gmail.com
https://kykungz.github.io/

When to Use GitHub

2 Situations + Special Case

Case 1: *You already have the project code on your local computer.*

Case 2: *Project exists on GitHub. You want to copy it to your computer.*

Special Case:

Case 3: *A new project (no files yet).*

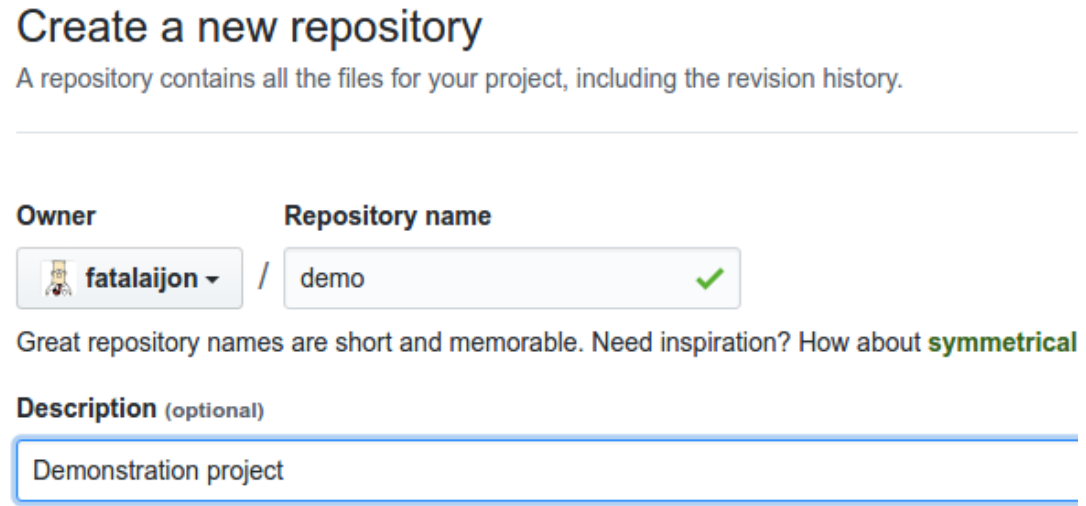
Case 1: Starting from Local Project

You already have a project on your computer

1. Create a local "git" repository.

```
cmd> git init
cmd> git add .gitignore src
cmd> git commit -m "initial code checkin"
```

2. Create an empty repo on GitHub.



Case 1: adding GitHub as a remote

3. Copy the URL of the new GitHub repository (https or ssh).

4. In your local project, add GitHub as a remote repository named "origin":

```
cmd> git remote add origin
     https://github.com/fatalaijon/demo.git
```

5. Copy the local repository to GitHub

```
cmd> git push -u origin master
```

You only need **"-u origin master"** the **first time** you push to GitHub.
Next time, just type "**`git push`**".

Case 2: Starting from GitHub

A project already exists on GitHub.
You want to "clone" it to your local computer & do work.

1. Get the GitHub project URL
   https://github.com/fatalaijon/demo.git

Or: go to the project on GitHub and click on 

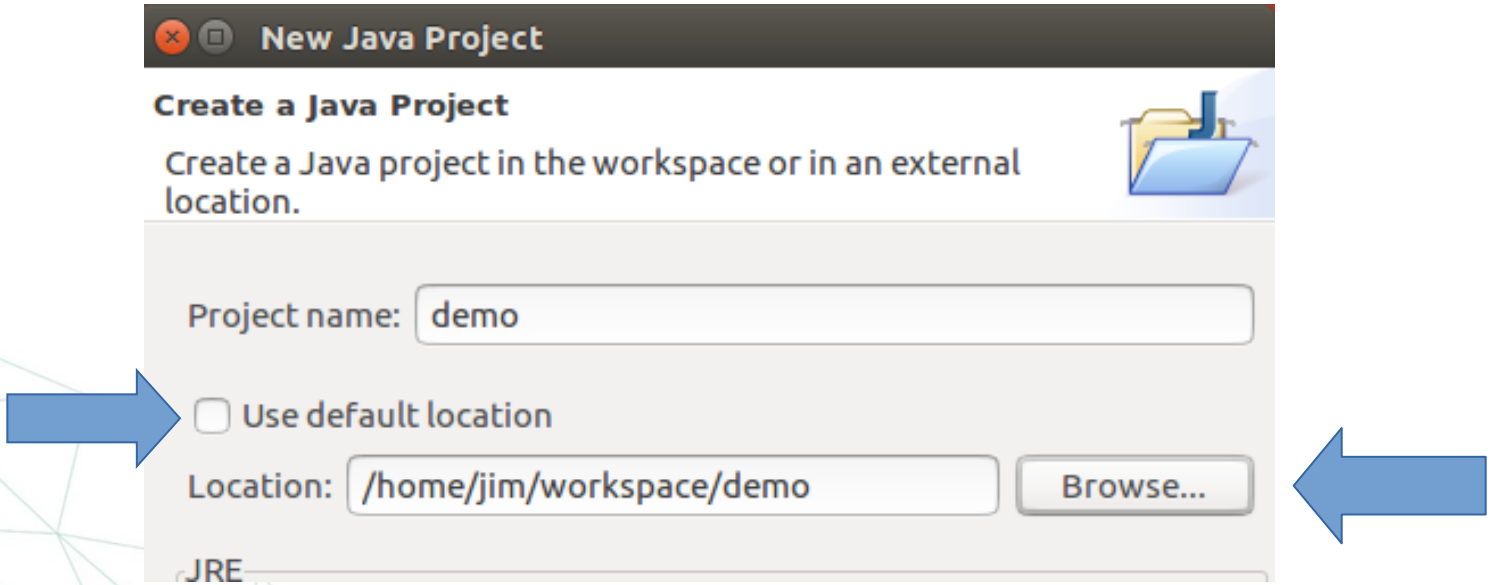
then copy the URL.

2. In your workspace, type:
   cmd> **git clone *https://github.com/...***

NOTE: "git clone" creates a new directory for the repository (named demo). If the directory already exists, clone won't work.

Case 2: Create an IDE project

3. Start your IDE and create a new project using the code in the directory you just cloned.



That's it!

GitHub is automatically the remote "`origin`".

Just "**`git push`**" your committed work to GitHub.

You can use a different project name

The name of your local directory (cloned from GitHub) can be different from the GitHub repository name.

1) Rename the directory yourself.

= or =

2) Specify the directory name when you "**clone**":

```
# Clone "demo" into local directory "mydemo"

cmd> git clone
https://github.com/fatalaijon/demo.git mydemo
```

Git Workflow

After you have a repository + GitHub, what do you do?

*Git workflow for an **individual** project:*

1) Check the status of your working copy:

```
cmd> git status
```

2) Commit changes or update your working copy.

"`git commit`" or "`git merge`"

3) Do some work:

Code, test. Code, test..... Review.

Now what?

Git Workflow (cont'd)

4) Check status again:

```
cmd> git status
Changes not staged for commit:
    modified: src/Problem2.java
Untracked files:
    src/Problem3.java
```

5) Add and commit your work to the local repository

```
cmd> git add src/Problem2.java src/Problem3.java
cmd> git commit -m "Solved problem 2 and 3"
[master 29abae0] Solved problems 2 and 3
2 files changed, 44 insertions(+), 5 deletions
```

Git Workflow (with GitHub)

6) Push changes to GitHub

```
cmd> git push
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 3.60 KiB, done.
Total 12 (delta 9), reused 0 (delta 0)
remote: Resolving deltas: 100% (9/9), ...
To https://github.com/fatailaijon/demo.git
 468abdf..29abae0  master -> master
```

That's it!

Repeat the cycle (1 - 6) as you work.

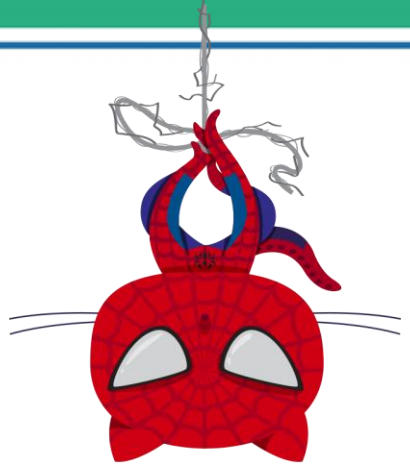
Git Workflow for Team Projects

On a **team project**, other people will be committing files to the GitHub repository.

For team projects, you should update your local repository from GitHub before trying to "push" your work to GitHub.

If GitHub updates your local repository, then you should merge changes into your working copy and test again, before trying to push to GitHub.

Reference



- http://en.wikipedia.org/wiki/Git_(software)
- http://ascc.sinica.edu.tw/iascc/articals.php?_section=2.4&_op=?articalID:4811
- http://coolshell.cn/articles/3288.html
- http://en.wikipedia.org/wiki/Revision_control
- http://billy3321.blogspot.tw/2009/02/github-howto.html

Quiz Time

رواد مصر الرقمية

Key Takeaways

وزارة الاتصالات وتكنولوجيا المعلومات

- **Branching:** Parallel development for features & fixes.
- **Merging:** Combine work from multiple branches.
- **Conflicts:** Natural part of teamwork, resolved manually.
- **Pull Requests:** Code review gateway to **main**.
- **Forking:** Copy someone’s repo for safe contributions.
- **CI/CD:** Automate testing & deployment using GitHub Actions.
- **Security:** Validate, update, and secure dependencies.
- **Rebasing:** Clean up commit history before merging.

رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

THANK YOU